

PYCOBOL LEXICAL ANALYZER

Course Title	Theory of Programming Language
Course Code	CT-367
Submitted To	Dr. Muhammad Kamran.
Language	PyCOBOL — Python + COBOL Hybrid
Group Members	MUHAMMAD DAIM UMER (CT-23089) ABDUL HADI (CT-23083) SYED HUMAIL MUSTAFA (CT-23082) UMER AMIR (CT-23056)
Section / Batch	Section B Batch 2023

DEPARTMENT OF COMPUTER SCIENCE & INFORMATION TECHNOLOGY
BACHELORS OF SCIENCE IN COMPUTER SCIENCE

Complex Computing Problem Assessment Rubrics

Course Code: CT-367		Course Title: Theory of Programming Language	
Criteria and Scales			
Excellent (3)	Good (2)	Average (1)	Poor (0)
<u>Criterion 1:</u> Understanding the Problem: How well the problem statement is understood by the student			
Understands the problem clearly and clearly identifies the underlying issues and functionalities.	Adequately understands the problem and identifies the underlying issues and functionalities.	Inadequately defines the problem and identifies the underlying issues and functionalities.	Fails to define the problem adequately and does not identify the underlying issues and functionalities.
<u>Criterion 2:</u> Research: The amount of research that is used in solving the problem			
Contains all the information needed for solving the problem	Good research, leading to a successful solution	Mediocre research which may or may not lead to an adequate solution	No apparent research
<u>Criterion 3:</u> Code: How complete the code is along with the assumptions and selected functionalities			
Complete Code according to the according to the selected functionalities of the given case with clear assumptions	Incomplete Code according to the selected functionalities of the given case with clear assumptions	Incomplete Code according to the selected functionalities of the given case with unclear assumptions	Wrong code and naming conventions
<u>Criterion 4:</u> Report: How thorough and well organized is the solution			
All the necessary information clearly organized for easy use in solving the problem	Good information organized well that could lead to a good solution	Mediocre information which may or may not lead to a solution	No report provided

Total Marks: _____

Teacher's Signature: _____

1. Introduction

1.1 What is a Lexical Analyzer?

A lexical analyzer, also known as a lexer or scanner, is the first phase of a compiler or interpreter. Its purpose is to read raw source code — which is simply a sequence of characters — and transform it into a structured stream of **tokens**. A token is a labeled, meaningful unit of the language, such as a keyword, an identifier, an operator, or a literal value. Without a lexer, a computer cannot begin to understand the structure or meaning of any program.

The lexical analysis phase is entirely separate from parsing or semantic analysis. It does not check grammar or meaning — it simply categorizes every word and symbol in the source code. Every programming language in existence — Python, C++, Java, SQL — has a lexer running as the first step of its compilation or interpretation pipeline. The programmer never sees it, but it is always there.

1.2 What is PyCOBOL?

PyCOBOL is a hybrid programming language designed and defined by this project. It combines two real, established languages: **COBOL** (Common Business-Oriented Language), a special-purpose language for business data processing, and **Python**, the leading modern general-purpose programming language. PyCOBOL allows a programmer to use COBOL's structured DIVISION/SECTION architecture, PIC data declarations, and business-oriented keywords alongside Python's if/else/for/while control flow and assignment syntax — all in a single source file.

1.3 Purpose of This Project

The purpose of this project is to design the PyCOBOL hybrid language and implement a complete lexical analyzer for it. The analyzer reads PyCOBOL source code and produces a token stream, a symbol table, a lexical error report, and a hybrid language score showing the ratio of COBOL to Python constructs in the analyzed program. An interactive web-based interface is also provided, built using Flask and JavaScript, allowing real-time analysis with a visual dashboard.

2. Background & Research

2.1 History of COBOL

COBOL — Common Business-Oriented Language — was created in **1959** under the direction of Dr. Grace Hopper, a United States Navy rear admiral and pioneer computer scientist. It was the first programming language designed specifically for business data processing, developed by a committee called CODASYL (Conference on Data Systems Languages) at the request of the U.S. Department of Defense.

COBOL was designed to be readable — almost like plain English. Commands like `MOVE "Ahmed Khan" TO EMPLOYEE-NAME` and `COMPUTE TAX = SALARY * TAX-RATE` were intentionally written so that business managers, not just programmers, could read and understand the code. This design philosophy made COBOL enormously successful in the business world of the 1960s and 70s.

2.2 Why COBOL Still Exists in 2026

Despite being over 65 years old, COBOL continues to power a significant portion of the world's financial infrastructure. The following facts illustrate its scale:

- There are an estimated **800 billion lines of COBOL** still in active production worldwide.
- COBOL processes over **\$3 trillion in daily commerce** — every ATM withdrawal, bank transfer, and payroll calculation.
- **95% of ATM transactions** globally run through COBOL-based systems.
- The **U.S. Social Security Administration, IRS, and major central banks** all operate on COBOL.
- Pakistan's major banks — HBL, MCB, UBL — run core banking systems on COBOL-based mainframes.

The reason COBOL persists is simple: **replacing it would cost more than keeping it**. The Commonwealth Bank of Australia spent **\$749 million and 5 years** just to partially migrate their core banking system. A full rewrite of a major bank's COBOL system would take a decade and risk catastrophic financial errors. So the code stays.

2.3 The COBOL Crisis

The critical problem facing the industry today is a **severe shortage of COBOL developers**. The average COBOL programmer is between 55 and 60 years old — they learned the language in the 1970s and 80s and are now retiring. Universities stopped teaching COBOL decades ago. Nobody young knows it.

This crisis became publicly visible during **COVID-19 in 2020**, when New Jersey's unemployment system crashed under demand. The system ran on COBOL. The governor appeared on television and **literally begged retired COBOL programmers to come out of retirement** to fix the system. IBM has since launched programs to train a new generation of COBOL developers, and universities are being paid to reintroduce COBOL into their curricula.

2.4 Real-World COBOL Modernization Efforts

The industry response to the COBOL crisis has been large-scale modernization — attempting to bridge legacy COBOL systems with modern languages, particularly Python. Real companies actively working on this include:

Company	Initiative
IBM	Built AI-powered tools to automatically analyze, document, and partially translate COBOL code to modern languages.
Deutsche Bank	Launched dedicated teams and projects to migrate COBOL-based financial systems to Python.
Accenture	Used GPT-4 to build a tool that generates COBOL technical documentation to accelerate modernization.
Microsoft	Partnered with Bankdata to develop open-source frameworks for COBOL code understanding.
Amazon AWS	Offers COBOL modernization as a managed cloud service to financial institutions.

2.5 Why Python + COBOL?

Python is the most widely used general-purpose programming language in the world today. It dominates data science, machine learning, web development, and automation. It is what the new generation of developers knows. COBOL is the special-purpose language that runs the financial world but is dying from a lack of developers.

The combination is therefore **directly relevant to the biggest modernization challenge in enterprise computing**. A lexical analyzer that understands both Python and COBOL syntax simultaneously is **the foundational first step** of any COBOL modernization pipeline. Before you can translate, migrate, or analyze COBOL code using modern tools, you must first be able to tokenize and understand it. That is exactly what this project builds.

3. PyCOBOL Language Design

3.1 Design Philosophy

PyCOBOL is designed around a single principle: **preserve COBOL's structured data architecture while replacing its cumbersome control flow with Python's modern, readable syntax**. COBOL excels at defining business data — its DIVISION/SECTION structure and PIC clauses are precise and expressive. Python excels at logic — its if/elif/else, for loops, and assignments are clean and intuitive. PyCOBOL takes the best of both.

3.2 Language Structure

A PyCOBOL program follows this structure:

```
IDENTIFICATION DIVISION
PROGRAM-ID: ProgramName
AUTHOR: Author Name

DATA DIVISION
WORKING-STORAGE SECTION
    01 VARIABLE-NAME      PIC 9(7)V99
    01 python_var          = "initial value"

PROCEDURE DIVISION
    * COBOL-style operations
    MOVE "Value" TO VARIABLE-NAME
    COMPUTE result = VARIABLE-NAME * 0.10

    # Python-style control flow
    if result > 5000:
        DISPLAY "High value"
    else:
        DISPLAY "Normal value"

    for i in range(10):
        DISPLAY "Iteration: " i

STOP RUN
```

3.3 Comparison: COBOL vs Python vs PyCOBOL

Feature	COBOL (Special Purpose)	Python (General Purpose)	PyCOBOL (Hybrid)
Data Declaration	PIC clauses (structured)	Dynamic typing	Both supported
Control Flow	EVALUATE/PERFORM (verbose)	if/for/while (clean)	Python syntax
Readability	English-like but verbose	Clean and minimal	Balanced
Business Domain	Excellent	Requires libraries	Excellent
Modern Features	Very limited	Full support	Via Python side
File Structure	DIVISION/SECTION	None	DIVISION/SECTION
Target Audience	Enterprise developers	All developers	Modernization teams

4. Selected Functionalities — Token Types

The PyCOBOL lexical analyzer recognizes the following token types. Each token type is defined by a regular expression rule and represents a distinct lexical category in the hybrid language.

Token Type	Category	Examples	Description
COBOL_DIVISION	COBOL	IDENTIFICATION DIVISION, DATA DIVISION, PROCEDURE DIVISION	Multi-word COBOL program structure headers
COBOL_SECTION	COBOL	WORKING-STORAGE SECTION, FILE SECTION	COBOL section declarations within divisions
COBOL_KEYWORD	COBOL	COMPUTE, MOVE, DISPLAY, PERFORM, ADD, STOP RUN	Reserved COBOL action words and level numbers
PYTHON_KEYWORD	Python	if, else, elif, for, while, def, return, True, False	Python reserved words (case-sensitive)
PICTURE_CLAUSE	COBOL	PIC A(30), PIC 9(7)V99, PIC 9V99	COBOL data type specifications
IDENTIFIER	Shared	employee_name, salary, net_pay, PayrollSystem	Variable names, function names, program names
STRING_LITERAL	Shared	"Ahmed Khan", "Engineering", 'CS Student'	Text values enclosed in single or double quotes
INTEGER	Shared	75000, 12, 0, 500	Whole number constants
FLOAT	Shared	0.10, 0.15, 90.50	Decimal number constants
OPERATOR	Shared	= + - * / == != <= >= < >	Arithmetic, assignment, and comparison operators
PUNCTUATION	Shared	() . , : []	Delimiters and structural characters
COMMENT	Both	* COBOL comment # Python comment	COBOL (* ...) and Python (# ...) style comments
UNCLOSED_STRING	Error	"Ahmed Khan (missing closing quote)	Error token: string literal not closed on same line
UNKNOWN	Error	@ \$ % ^ ~	Error token: character not part of PyCOBOL alphabet

5. Regular Expressions & Rules

The lexer is built using Python's `re` module. All token rules are compiled into a single master regular expression using named groups, which is applied to the source code in a single left-to-right pass. Rules are ordered from **most specific to most general** to prevent shorter patterns from consuming longer ones.

Token Type	Regular Expression	Notes
COMMENT	<code>*.* #.*</code>	Matches from <code>*</code> or <code>#</code> to end of line. COBOL uses <code>*</code> , Python uses <code>#</code> .
UNCLOSED_STRING	<code>"[^"\n]*\$ \'[^\'\n]*\$</code>	Matches quote opened but not closed before newline. Error token.
STRING_LITERAL	<code>"[^"]*" \'[^\']*\'</code>	Matches any text between matching quote characters.
PICTURE_CLAUSE	<code>\bPIC\s+[A-Za-z0-9().V\-\-]+\b</code>	Matches PIC keyword followed by type spec: PIC 9(7)V99, PIC A(30).
FLOAT	<code>\b\d+\.\d+\b</code>	One or more digits, decimal point, one or more digits.
INTEGER	<code>\b\d+\b</code>	One or more decimal digits. Must come after FLOAT.
OPERATOR	<code>== != <=> < > [=+\-*/]</code>	Two-character operators checked before single-char operators.
PUNCTUATION	<code>[().,:;[\]]</code>	Any single punctuation character in the defined set.
IDENTIFIER	<code>\b[A-Za-z_][A-Za-z0-9_\-]*\b</code>	Starts with letter or underscore. Hyphens allowed for COBOL names.
WHITESPACE	<code>[\t]+</code>	Spaces and tabs. Silently discarded after tokenization.
NEWLINE	<code>\n</code>	Used for line number tracking only. Not emitted as a token.
UNKNOWN	<code>.</code>	Catch-all: any character not matched above. Triggers error.

5.1 Keyword Classification Logic

Because PyCOBOL contains keywords from two languages, collision handling is required. For example, the word `IF` exists in both COBOL (uppercase) and Python (lowercase `if`). The lexer resolves this using exact case matching: if the matched identifier is lowercase and appears in the Python keyword set, it is classified as `PYTHON_KEYWORD`. If it is uppercase and appears in the COBOL keyword set, it is classified as `COBOL_KEYWORD`. All COBOL keywords are automatically normalized to uppercase in the output.

5.2 Multi-Word Token Handling

COBOL contains multi-word tokens such as IDENTIFICATION DIVISION, WORKING-STORAGE SECTION, and STOP RUN. Because the master regex processes one token at a time, these phrases are handled using a **pre-pass substitution**: before the main scan, all known multi-word phrases are temporarily replaced with single-word placeholders (e.g., IDENTIFICATION__SP__DIVISION). After matching, placeholders are restored to their original values and assigned the correct token type.

6. Source Code

The project consists of three main source files. **lexer.py** contains the core lexical analyzer engine. **app.py** is a Flask web server that exposes the lexer via a REST API. **script.js** and **style.css** implement the interactive browser-based frontend.

6.1 Project File Structure

```
pycobol-analyzer/
■■■ lexer.py           ← Core lexical analyzer (tokenizer + symbol table + errors)
■■■ app.py             ← Flask web server (REST API)
■■■ templates/
■   ■■■ index.html     ← Web interface HTML
■■■ static/
■   ■■■ style.css       ← Frontend styling
■   ■■■ script.js       ← Frontend logic (dashboard, tables, charts)
■■■ tests/
■   ■■■ test_input.pycobol    ← Payroll system test
■   ■■■ test_clean.pycobol    ← Student grading test (no errors)
■   ■■■ test_errors.pycobol   ← Intentional error test
■   ■■■ test_all_tokens.pycobol ← Full token coverage test
■   ■■■ test_mixed_errors.pycobol ← Mixed error recovery test
■   ■■■ test_unclosed_strings.pycobol
■   ■■■ test_unknown_chars.pycobol
```

6.2 lexer.py — Core Tokenizer

```
TOKEN_RULES = [
    ("COMMENT",      r'\*.*|#.*'),
    ("UNCLOSED_STRING", r'("[^"\n]*$|\'[^\'\n]*$)'),
    ("STRING_LITERAL", r'"([^\"]*)"|\'([^\']*)*\''),
    ("PICTURE_CLAUSE", r'\bPIC\s+[A-Za-z0-9().V\-\]+\b'),
    ("FLOAT",         r'\b\d+\.\d+\b'),
    ("INTEGER",        r'\b\d+\b'),
    ("OPERATOR",       r'==|!=|<=|>=|<|>|[\+\-\*/]'),
    ("PUNCTUATION",    r'[(\),;\[\]]'),
    ("IDENTIFIER",     r'\b[A-Za-z_][A-Za-z0-9_\-]*\b'),
    ("WHITESPACE",     r'[\t]+'),
    ("NEWLINE",        r'\n'),
    ("UNKNOWN",        r'.'),
]

MASTER_REGEX = re.compile(
    '|'.join(f'(?P{pattern})'
            for i, (_, pattern) in enumerate(TOKEN_RULES)),
    re.IGNORECASE | re.MULTILINE
)
```

6.3 Symbol Table Builder

```
def build_symbol_table(tokens):
    symbols = {}
    current_scope = "GLOBAL"

    # Tracks scope as divisions are encountered
    if tok.token_type == "COBOL_DIVISION":
        current_scope = tok.value # e.g. "DATA DIVISION"

    # COBOL variable: 01 var_name PIC 9(7)V99
```

```

if tok.token_type == "COBOL_KEYWORD" and tok.value in ("01","05"...):
    var_name = tokens[i+1].value
    data_type = tokens[i+2].value # PIC clause

# Python assignment: salary = 75000
if tok.token_type == "IDENTIFIER" and next_tok.value == "=":
    inferred_type = infer_type(tokens[i+2])

# Function definition: def func_name
if tok.token_type == "PYTHON_KEYWORD" and tok.value == "def":
    entry_type = "FUNCTION"

return symbols

```

6.4 Flask API — app.py

```

@app.route("/analyze", methods=["POST"])
def analyze():
    source_code = request.get_json()["source_code"]
    tokens, errors = tokenize(source_code)
    symbols = build_symbol_table(tokens)

    # Calculate hybrid score
    cobol_count = sum(COBOL token counts)
    python_count = counts.get("PYTHON_KEYWORD", 0)
    cobol_pct = cobol_count / (cobol_count + python_count) * 100

    return jsonify({
        "tokens": token_list,
        "errors": error_list,
        "symbols": symbol_list,
        "analysis": { "cobol_pct": cobol_pct, ... }
    })

```

7. Sample Input & Output

7.1 Sample PyCOBOL Program — Payroll System (test_input.pycobol)

```
* PyCOBOL Sample Program – Payroll Processing System
IDENTIFICATION DIVISION
PROGRAM-ID: PayrollSystem
AUTHOR: CS Student

DATA DIVISION
WORKING-STORAGE SECTION
    01 employee_name    PIC A(30)
    01 salary            PIC 9(7)V99
    01 tax_rate          PIC 9V99
    01 bonus             PIC 9(5)V99
    01 department        PIC A(20)

PROCEDURE DIVISION
employee_name = "Ahmed Khan"
salary = 75000
tax_rate = 0.10

COMPUTE bonus = salary * 0.15
COMPUTE tax = salary * tax_rate

if salary > 50000:
    DISPLAY "Employee is in high salary bracket"
    COMPUTE net_pay = salary + bonus - tax
else:
    COMPUTE net_pay = salary - tax

for month in range(12):
    DISPLAY "Processing month: " month

STOP RUN
```

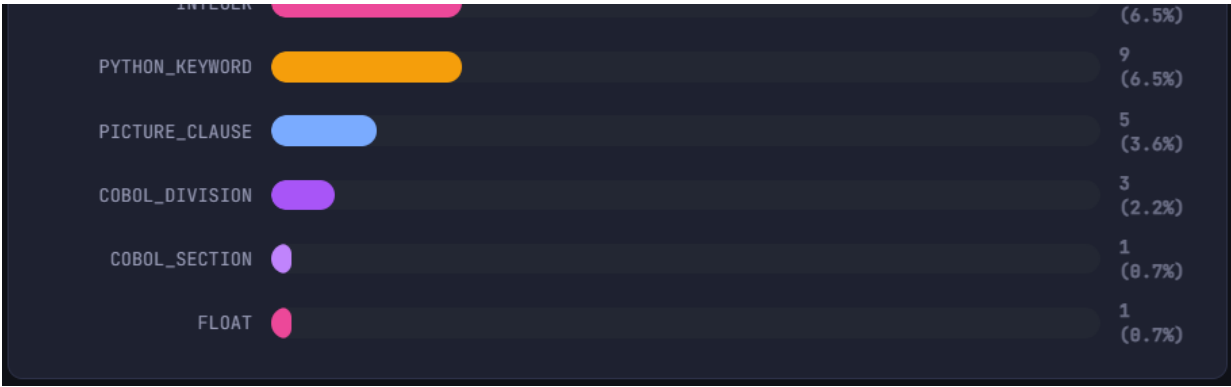
7.2 Token Stream Output (partial — 139 tokens total)

#	TOKEN TYPE	VALUE	LINE	COL
1	COMMENT	* PyCOBOL Sample Program – Payroll...	1	1
2	COMMENT	* Demonstrates hybrid COBOL + Python syntax	2	1
3	COBOL_DIVISION	IDENTIFICATION DIVISION	4	1
4	COBOL_KEYWORD	PROGRAM-ID	5	1
5	PUNCTUATION	:	5	10
6	IDENTIFIER	PayrollSystem	5	12
7	COBOL_KEYWORD	AUTHOR	6	1
8	COBOL_DIVISION	DATA DIVISION	8	1
9	COBOL_SECTION	WORKING-STORAGE SECTION	9	1

10	COBOL_KEYWORD	01	10	5
11	IDENTIFIER	employee_name	10	8
12	PICTURE_CLAUSE	PIC A(30)	10	24
13	COBOL_KEYWORD	01	11	5
14	IDENTIFIER	salary	11	8
15	PICTURE_CLAUSE	PIC 9(7)V99	11	24
16	COBOL_DIVISION	PROCEDURE DIVISION	16	1
17	IDENTIFIER	employee_name	19	1
18	OPERATOR	=	19	15
19	STRING_LITERAL	"Ahmed Khan"	19	17
20	IDENTIFIER	salary	20	1
21	OPERATOR	=	20	8
22	INTEGER	75000	20	10
23	COBOL_KEYWORD	COMPUTE	25	1
24	IDENTIFIER	bonus	25	9
25	OPERATOR	=	25	15
26	IDENTIFIER	salary	25	17
27	OPERATOR	*	25	24
28	FLOAT	0.15	25	26
29	PYTHON_KEYWORD	if	29	1
30	IDENTIFIER	salary	29	4
...	...	...	...	...
139	COBOL_KEYWORD	STOP RUN	56	1

7.3 Token Summary — test_input.pycobol





7.4 Hybrid Language Score

Metric	Value
COBOL Constructs (keywords + divisions + sections + PIC)	28 tokens
Python Constructs (keywords)	9 tokens
COBOL Percentage	75.7%
Python Percentage	24.3%
Hybrid Score Bar	[#####.....] 76% COBOL / 24% Python
Interpretation	Predominantly COBOL structure with Python control flow — typical for a migrating enterprise program

8. Symbol Table Output

The symbol table is built in a second pass over the token stream after tokenization is complete. It records every declared variable, its type, its inferred data type, its initial value (if assigned), its scope (which DIVISION it was declared in), and the line number of its first occurrence.

8 symbols

#	NAME	TYPE	DATA TYPE	VALUE	SCOPE	LINE
1	PayrollSystem	PROGRAM	PROGRAM-ID	--	IDENTIFICATION DIVISION	5
2	employee_name	VARIABLE	STRING	"Ahmed Khan"	PROCEDURE DIVISION	19
3	salary	VARIABLE	INTEGER	75000	PROCEDURE DIVISION	20
4	tax_rate	VARIABLE	FLOAT	0.10	PROCEDURE DIVISION	21
5	department	VARIABLE	STRING	"Engineering"	PROCEDURE DIVISION	22
6	bonus	VARIABLE	--	salary	PROCEDURE DIVISION	25
7	tax	VARIABLE	--	salary	PROCEDURE DIVISION	26
8	net_pay	VARIABLE	--	salary	PROCEDURE DIVISION	31

Note on Scope Tracking: The symbol table builder tracks which COBOL DIVISION is currently active by monitoring COBOL_DIVISION tokens. Variables declared under DATA DIVISION receive scope DATA DIVISION, while assignments made during PROCEDURE DIVISION execution receive scope PROCEDURE DIVISION. The program name declared after PROGRAM-ID receives scope IDENTIFICATION DIVISION.

9. Error Detection Output

The PyCOBOL lexer includes three types of lexical error detection. Importantly, the lexer implements **error recovery** — when an error is found, it is recorded and scanning continues. This allows the lexer to report all errors in a file in a single pass, rather than stopping at the first error.

Error Type	Trigger	Example
UNCLOSED_STRING	A string literal opened with a quote but not closed before end of line	employee_name = "Ahmed Khan ← missing closing "
UNKNOWN_CHAR	Any character not in the PyCOBOL alphabet	total = salary + bonus @ tax ← @ is invalid
INVALID_PIC	A PIC clause contains characters outside the valid set (9, A, X, V, S, B, Z)	01 bad_pic PIC Z(5) ← Z is not a valid PIC type

9.1 Error Test Input (test_errors.pycobol)

```
IDENTIFICATION DIVISION
PROGRAM-ID: ErrorTest

DATA DIVISION
WORKING-STORAGE SECTION
    01 employee_name    PIC A(30)
    01 salary           PIC 9(7)V99
    01 bad_pic          PIC Z(5)      ← INVALID_PIC error

PROCEDURE DIVISION
employee_name = "Ahmed Khan      ← UNCLOSED_STRING error
salary = 75000

total = salary + bonus @ tax      ← UNKNOWN_CHAR error (@)
price = 100 $ 50                  ← UNKNOWN_CHAR error ($)

DISPLAY "Hello World              ← UNCLOSED_STRING error

COMPUTE bonus = salary * 0.15
DISPLAY "This is fine"
STOP RUN
```

9.2 Error Report Output

UNCLOSED_STRING Line 18, Col 9
String literal is missing closing quote
"Ahmed Khan

UNKNOWN_CHAR Line 22, Col 15
Unexpected character '@'
@

UNKNOWN_CHAR Line 23, Col 15

Unexpected character '\$'

\$

UNCLOSED_STRING Line 26, Col 7

String literal is missing closing quote

"Hello World

10. Web Interface

In addition to the command-line interface, the project includes a full web-based interactive analyzer built with Flask (Python backend) and vanilla JavaScript (frontend). The interface provides a real-time analysis dashboard that is significantly more user-friendly than terminal output.

10.1 Features

- **Source Code Editor** — Syntax-aware text area with live line numbers and Ctrl+Enter shortcut to analyze.
- **Test File Loader** — Dropdown to load any .pycobol test file directly from the server.
- **Dashboard Tab** — Animated stat cards (Lines, Tokens, Symbols, Errors) and animated hybrid score bar.
- **Token Stream Tab** — Full searchable and filterable token table with type, value, line, and column.
- **Symbol Table Tab** — Displays all declared variables with type, data type, value, scope, and line.
- **Errors Tab** — Color-coded error cards with error type, location, message, and offending fragment.
- **Token Distribution Chart** — Animated horizontal bar chart showing all token type counts.

10.2 How to Run

```
# Install dependencies
pip install flask

# Run the web server
python app.py

# Open in browser
http://127.0.0.1:5000
```

10.3 API Endpoints

Endpoint	Method	Description
/ (index)	GET	Serves the main HTML interface with test file dropdown
/analyze	POST	Accepts source_code JSON, returns tokens, errors, symbols, analysis
/load_file	POST	Accepts filename JSON, returns contents of a .pycobol test file

10.4 User Interface



PyCOBOL Lexical Analyzer

Theory of Programming Language (CT-367)

SOURCE CODE

33 lines

1

* PyCOBOL Error Test File â€” Tests Lexical Error Detection

2

* This file intentionally contains lexical errors

3

4

IDENTIFICATION DIVISION

5

PROGRAM-ID: ErrorTest

6

AUTHOR: CS Student

7

8

DATA DIVISION

9

WORKING-STORAGE SECTION

10

01 employee_name PIC A(30)

11

01 salary PIC 9(7)V99

12

01 bad_pic PIC Z(5)

13

01 department PIC A(20)

14

15

PROCEDURE DIVISION

16

17

Test 1: Unclosed string literal

18

employee_name = "Ahmed Khan

19

salary = 75000

20

21

Test 2: Unknown/invalid characters

22

total = salary + bonus @ tax

23

price = 100 \$ 50

24

25

Test 3: Another unclosed string

26

DISPLAY "Hello World

27

28

Test 4: Valid code to show recovery works

29

COMPUTE bonus = salary * 0.15

30

DISPLAY "This is fine"

31

32

STOP RUN

test_errors.pycobol

Analyze

Clear

Dashboard

Tokens

Symbols

Errors 4

Search tokens...

All Types

58 tokens

#	TOKEN TYPE	VALUE	LINE	COL
42	IDENTIFIER	tax	22	17
43	IDENTIFIER	price	23	3
44	OPERATOR	=	23	9
45	INTEGER	100	23	11
46	INTEGER	50	23	17
47	COMMENT	# Test 3: Another unclosed string	25	21
48	COBOL_KEYWORD	DISPLAY	26	21
49	STRING_LITERAL	"Hello World	26	7
50	COMMENT	# Test 4: Valid code to show recovery works	28	21
51	COBOL_KEYWORD	COMPUTE	29	21
52	IDENTIFIER	bonus	29	29
53	OPERATOR	=	29	5
54	IDENTIFIER	salary	29	7
55	COMMENT	* 0.15	29	14
56	COBOL_KEYWORD	DISPLAY	30	21
57	STRING_LITERAL	"This is fine"	30	5
58	COBOL_KEYWORD	STOP RUN	32	12

11. Assumptions & Limitations

11.1 Assumptions

- PyCOBOL source files use UTF-8 encoding.
- COBOL keywords are case-insensitive (COMPUTE = compute = Compute). Python keywords are case-sensitive (if ≠ IF).
- COBOL identifiers may use hyphens (EMPLOYEE-NAME). Python identifiers use underscores (employee_name).
- COBOL DIVISION and SECTION headers appear on their own lines.
- Multi-word COBOL phrases (STOP RUN, WORKING-STORAGE SECTION) are handled via pre-pass substitution.
- PIC clauses use formats: PIC 9(n), PIC A(n), PIC X(n), PIC 9(n)V99.
- Both comment styles are supported: COBOL (* ...) and Python (# ...).
- The lexer performs tokenization only — no parsing, semantic analysis, or code execution.

11.2 Limitations

- The lexer does not perform syntax or grammar validation. Structurally invalid PyCOBOL will still be tokenized.
- COBOL COPY statements and file inclusion are not supported.
- Multi-line strings are not supported — strings must open and close on the same line.
- Nested quotes within string literals are not handled.
- Complex COBOL PIC patterns with sign symbols (S9(5)V99) may not be fully captured.
- The symbol table does not handle variable shadowing or nested scopes beyond DIVISION-level tracking.
- The lexer does not validate that COBOL-style variables declared in DATA DIVISION are used correctly in PROCEDURE DIVISION.
- Error recovery is basic — scanning continues after errors but no correction is attempted.

12. Future Work

The lexical analyzer is the first phase of a compiler. The natural next steps to extend this project into a more complete language processing system are:

- **Phase 2 — Parser:** Build a syntax parser for PyCOBOL that checks the grammatical structure of the token stream and produces an Abstract Syntax Tree (AST). This is the logical next step in compiler construction.
- **Phase 3 — Semantic Analyzer:** Check type compatibility, verify that variables are declared before use, and validate COBOL PIC clause usage against assigned values.
- **Phase 4 — Code Generator:** Translate PyCOBOL's AST into executable Python code, effectively creating a COBOL-to-Python transpiler — directly applicable to the real-world modernization problem described in Section 2.
- **COBOL Dialect Support:** Extend token rules to support COBOL-85, COBOL-2002, and COBOL-2014 dialects, increasing compatibility with real enterprise codebases.
- **IDE Plugin:** Package the lexer as a VS Code or JetBrains plugin to provide real-time syntax highlighting and error detection for PyCOBOL files.
- **AI-Assisted Migration:** Integrate the lexer with a large language model to automatically suggest Python equivalents for COBOL constructs, building on the same approach used by Accenture's COBOL modernization tool.

Industry Relevance: Deutsche Bank, IBM, Microsoft, Accenture, and Amazon AWS are all currently investing heavily in exactly this pipeline — from lexical analysis to full automated COBOL modernization. This project, while academic in scale, is directly aligned with one of the most expensive and urgent technical challenges facing the global financial industry today.

13. References

- [1] Aho, A. V., Lam, M. S., Sethi, R., & Ullman, J. D. (2006). *Compilers: Principles, Techniques, and Tools* (2nd ed.). Addison-Wesley. [The Dragon Book — standard compiler textbook]
- [2] IBM Corporation. (2023). *IBM COBOL Migration and Modernization*. IBM Documentation.
<https://www.ibm.com/docs/en/cobol-zos>
- [3] Brodie, M. L. (2019). The Challenges and Opportunities in Modernizing Legacy COBOL Systems. *IEEE Software*, 36(5), 12–15.
- [4] Python Software Foundation. (2024). *Python 3 re — Regular Expression Operations*.
<https://docs.python.org/3/library/re.html>